

Um pouco da história da computação:

- No início funcionários eram chamados de computadores, e os que tiravam e colocavam cabos nas máquinas de maneira manual durante a execução de um programa para que a máquina seja efetuada de maneira correta.

- Tipos de compiladores:
 - Interpretada: Um "interpretador" lê o seu código durante a execução, não faz nenhum pré-compilação, compila durante a execução do código.

Principal vantagem: O programa pode ser executado em diferentes sistemas, pois compila em tempo real para linguagem de máquina do sistema.

Principal Desvantagem: O processo completo de execução é mais demorado pois o código compila enquanto está executando.

- Compilação Estática:

O código é inteiro
parado para linguagens
de máquina, só depois
que é executado.

Principal vantagem: A máquina
executa o código já compi-
lado e que deixa o processo
mais rápido.

Principal desvantagem: O executável
só irá funcionar no mesmo
tipo de sistema em que foi
compilado.

- Compilação Híbrida:

Primeiro faz a compilação estática para um bytecode (não o código de máquina direto). Depois o bytecode é interpretado durante a execução.

Principal vantagem: Consegue ser mais flexível quando se trata de desempenho e portabilidade pois é "pré compilada" assim facilitando a tradução para o interpretador.

Principal desvantagem: É mais complexo que modelos puros e pode ocasionar problemas mais difíceis de solucionar.

Tipos de compiladores:

- Em geral são 3 tipos

Análises:

- Lexica
- Sintática
- Semântica

Otimização:

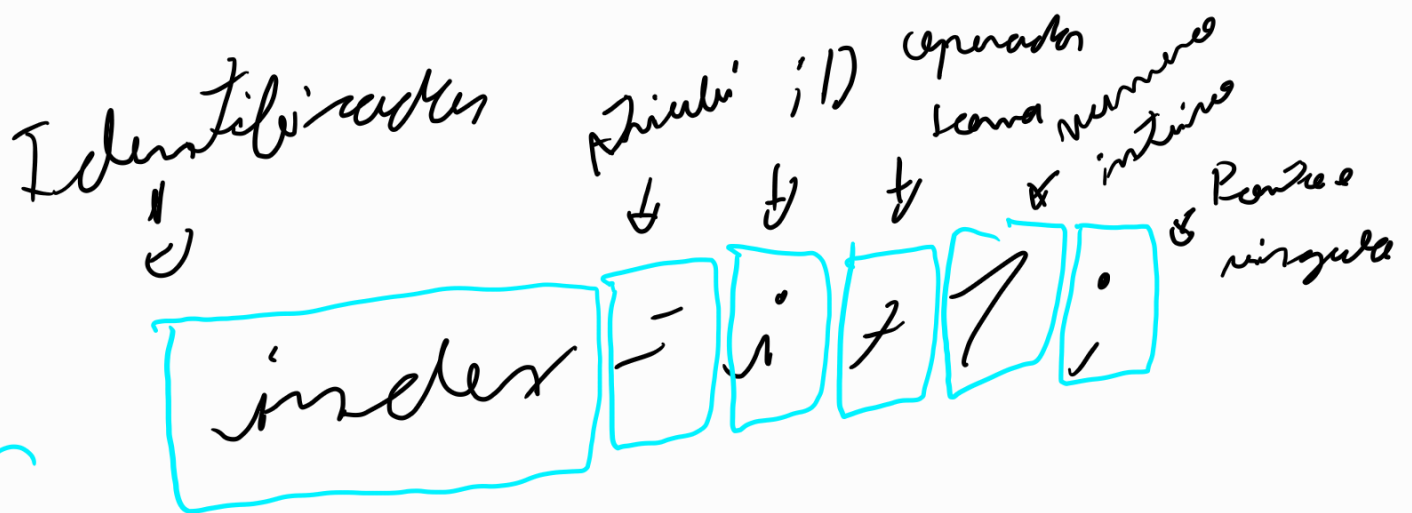
- Otimizadores
de código.

Geradores:

- Geradores de
código.

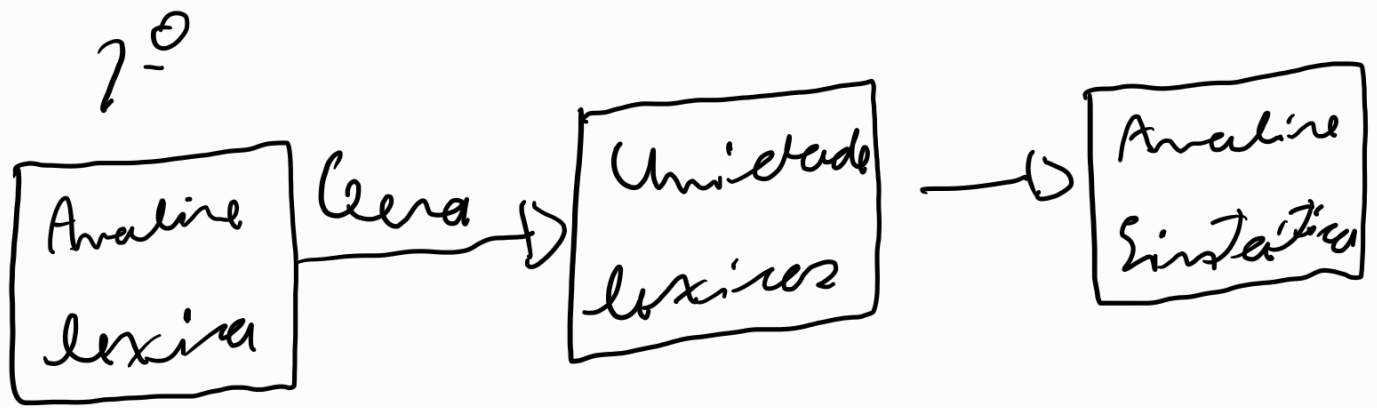
Análise Lexical:

- Separa os limites de código em tokens.
- "Pega um texto e divide, mas não interpreta nada"



→ Está fazendo os caracteres dos tokens.

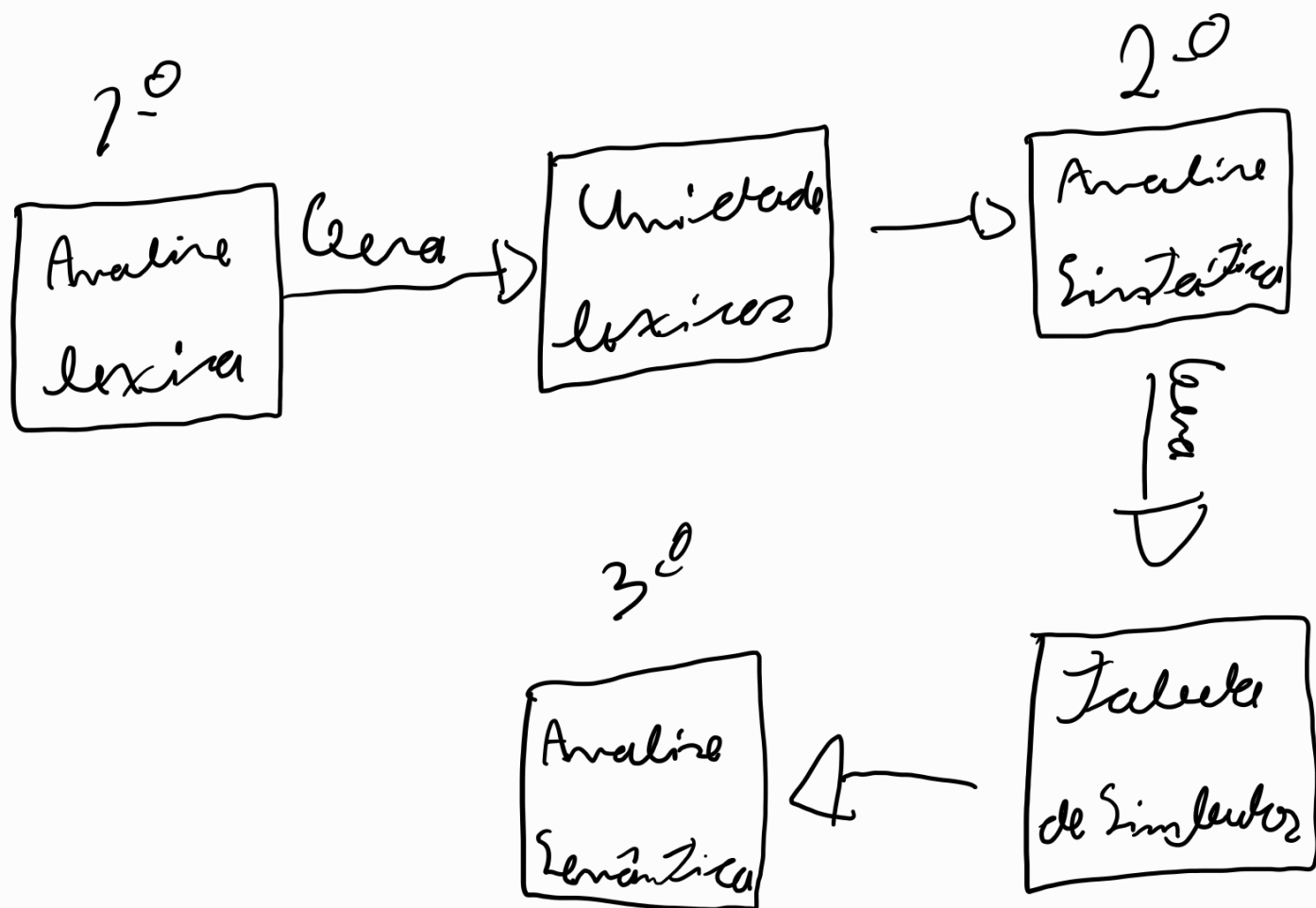
Análise Sintática:



- A análise sintática
verifica se a ordem dos
tokens/Unidade Lexical
está na ORDEM certa.

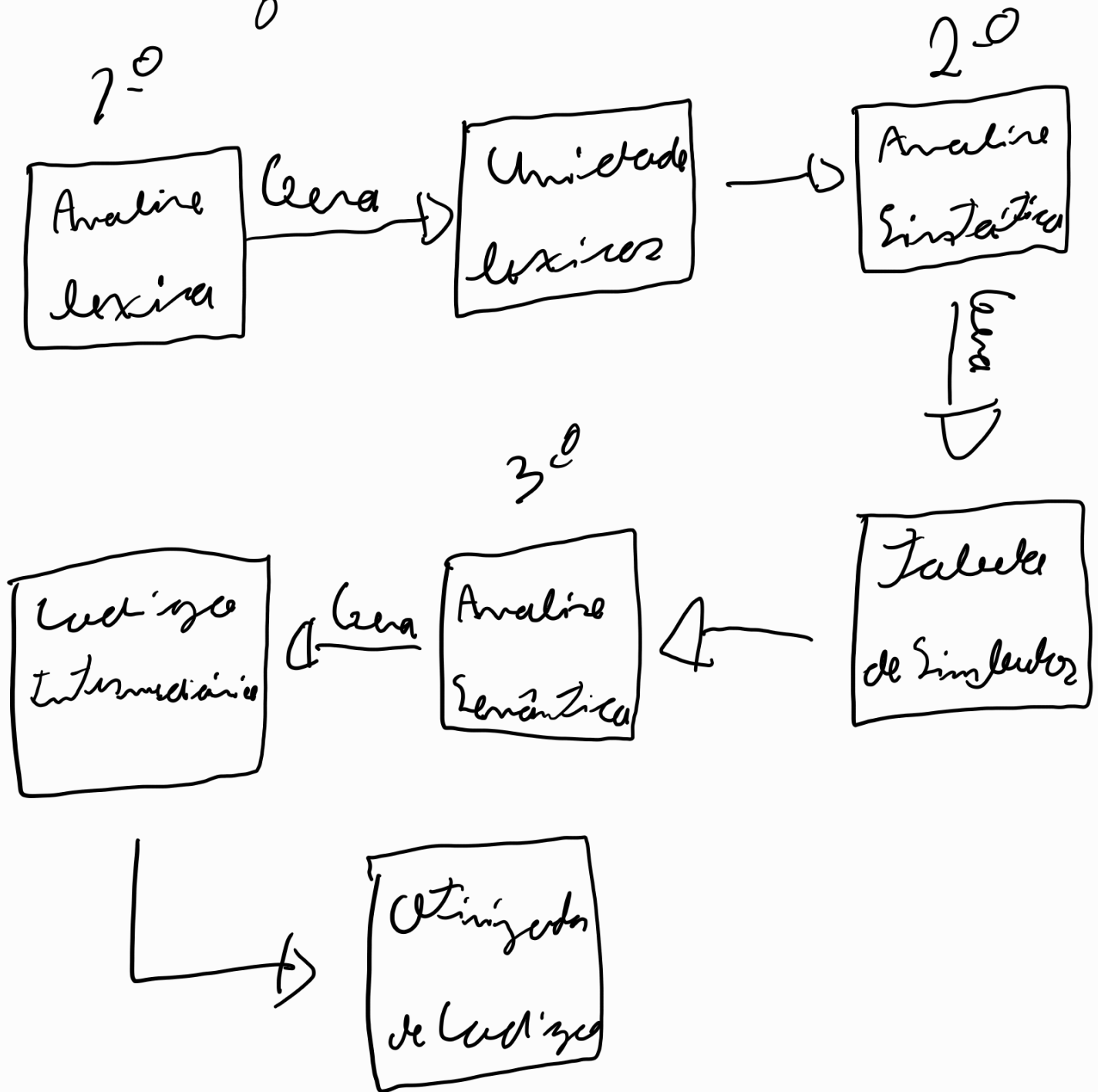
- "verifica apenas a
ordem dos tokens"

Análise Semântica:



- A análise Semântica verifica se os tipos das variáveis estão corretos.
- Faz verificações de estrutura.

Organização de Códigos:



“Não vemos mais
disciplina”

• Gramática:

- No português o sujeito
não se pode ser separado
por “,” de um verbo.

Arrimamos em factos
as linguagens de cálculo
as linguagens de programação
também tem suas regras gramaticas:

“Cada linguagem tem sua
sintaxe e sua lógica”

• Gramática BNF

"Badeus Noun Form"

↳ "Uma linguagem para explicar linguagens"

OBS: A dor's símbolos que se referem a linguagens:

""

OBS: O problema não é uma mistura dos dois para ficar mais fácil entender da "meio"

BNF:

Exemplos de gramáticas simples:

Regras da gramática:
(ou metagramática)

$\langle \text{digBin} \rangle \rightarrow 0$ $\rightarrow \text{digBin}$ é um
não terminal

$\langle \text{digBin} \rangle \rightarrow 1$

$\langle \text{digBin} \rangle \rightarrow 0 \langle \text{digBin} \rangle$

$\langle \text{digBin} \rangle \rightarrow 1 \langle \text{digBin} \rangle$

- Esta simples gramáticas

tem 4 regras! Sendo possíveis

expressões infinitas 0 e 1!

É possível também
escrever as mesmas regras
de outra maneira:

Regras da gramática:
(ou metagramática)

$\langle \text{digBin} \rangle = 00 \underset{\text{ou}}{1} \underset{\text{ou...}}{1} 0 \langle \text{digBin} \rangle / 1 \langle \text{digBin} \rangle$

Dessa maneira podemos
verificar uma demonstração:

- Como provamos que a
string 011 faz parte
desta gramática?

* Para responder a essa pergunta devemos usar uma demonstração com pares e pares:

③ $\rightarrow 0 \text{ (digBin)}$ $\rightarrow 007 \text{ (digBin)}$
② $\rightarrow 077$

6) Pronto para provar que por meio de derivações que a string 077 é de fato da gramática apresentada.

OBS: Provamos por meio
de derivação, fizemos
uma derivação da linguagem
para provarmos que de fato obedecia
aquela gramática.

• Lista de exercícios :

Universidade Federal do Ceará – Campus Russas
Linguagens de Programação
Prof. Ms. Rafael Ivo

Lista de Exercícios 1 – Introdução

- 1) Cite áreas de aplicação de programação e linguagens que exemplifiquem essas áreas.
- 2) O que é a sintaxe de uma linguagem de programação?
- 3) O que é a semântica de uma linguagem de programação?
- 4) O que é um paradigma de programação?
- 5) Cite e diferencie os paradigmas de programação existentes nas linguagens de programação atuais.
- 6) Porque a arquitetura de computadores influencia no projeto de criação de uma nova linguagem de programação?
- 7) Porque simplicidade é um dos critérios mais importantes na avaliação de uma linguagem de programação?
- 8) O que significa dizer que uma linguagem de programação é confiável?
- 9) O que significa abstração em uma linguagem de programação?
- 10) O que significa ortogonalidade de uma linguagem de programação? Por que, ortogonalidade em excesso também pode ser um problema?
- 11) Explique a diferença entre: linguagens compiladas, linguagens interpretadas puras e linguagens interpretadas híbridas.
- 12) Relacione as etapas realizadas durante o processo de compilação de um determinado programa.

7) Área de inteligência artificial,
área de comércio, área da saúde.

2) A sintaxe é usada para verificar
se os dados dos textos lexiais
estão corretos.

3) A semântica verifica se os tipos de variáveis estão declarados de maneira correta, como int, float, etc.

4) Paradigmas da programação não um conjunto de regras de como a linguagem vai executar o programa através do que foi programado, a ordem de execução, a ordem de como os códigos devem ser lidos não tendo a orientação para paradigma que a linguagem segue.

5)

6/ Inclusive pois existem
muitos tipos de ritmos,
Hendecavos e rebatuas já,
cuidado por outros linguagens
então é muito importante
pensar nos padrões de mercado
que já são por demonstrar.

7/ Para que fique mais fácil
de ser desenvolvida, aprendida,
ensinada e para que de
maneira mais fácil seja possível
ter manutenção nos ritmos certos.

8) Significa que a linguagem segue todos os parâmetros estabelecidos pelas convenções determinadas.

9) Abstração é um ato de imaginar, abstrair algo ou ideia que já é presente na realidade e transformá-la em "tradição", para a fase de propagação

10 /

77 / linguagem compilada
O código é produzido inteiramente primeiro para linguagem de máquina e depois posteriormente é executado, já na interpretação o código vai sendo compilado linha após linha durante a presença de execução "em tempo real" já a biblioteca compila o código primeiro para um bytecode e posteriormente em interpretados vai traduzindo o bytecode para linguagem de máquina.

7.1.1 Temos 5 etapas em
principal, 1.^o a lexica, 2.^o a análise
sintática, depois da semântica,
após temos a otimização e o
gerador de código. O compilador
segue essas etapas para a par-
te de como executar o res-
ultado para que o código
não compile constantemente.

Derivação à Esquerda

Regras da gramática:

^{1) são não terminais}
 $\langle int \rangle \rightarrow \langle dig \rangle \langle int \rangle \mid \langle dig \rangle$
 $\langle dig \rangle \rightarrow 0 \mid 1 \mid 2 \mid \dots \mid 9$

Derivação:

$\langle int \rangle$

$\langle dig \rangle \langle int \rangle$

7 $\langle int \rangle$

7 $\langle dig \rangle \langle int \rangle$

7 0 $\langle int \rangle$

7 0 $\langle dig \rangle$

7 0 7

Prova

A derivação

sempre segue

à esquerda

sem resto

derivando de

pois a parte

resta esquerda.

Permanece a direita,

$\langle int \rangle$

$\langle dig \rangle \underline{\langle int \rangle}$

Fazer depois para obter

Fixações:

+ São não determinais

$\langle \text{exp} \rangle \rightarrow \langle \text{exp} \rangle + \langle \text{ter} \rangle \mid \langle \text{exp} \rangle * \langle \text{ter} \rangle \mid \langle \text{ter} \rangle$

$\langle \text{ter} \rangle \rightarrow 0 \mid 1 \mid 2 \mid 3 \dots \mid 9 \mid (\langle \text{exp} \rangle)$

Prove:

$(2 + 2) * 3$

$\langle \text{exp} \rangle$

$\langle \text{exp} \rangle * \langle \text{ter} \rangle$

$\langle \text{ter} \rangle * \langle \text{ter} \rangle$

$\langle \langle \text{exp} \rangle \rangle * \langle \text{ter} \rangle$

$(\langle \text{exp} \rangle + \langle \text{ter} \rangle) * \langle \text{ter} \rangle$

$(\langle \text{ter} \rangle + \langle \text{ter} \rangle) * \langle \text{ter} \rangle$

$(2 + \langle \text{ter} \rangle) * \langle \text{ter} \rangle$

$(2 + 2) * \langle \text{ter} \rangle$

$(2 + 2) * 3 \rightarrow \text{Provedo}$

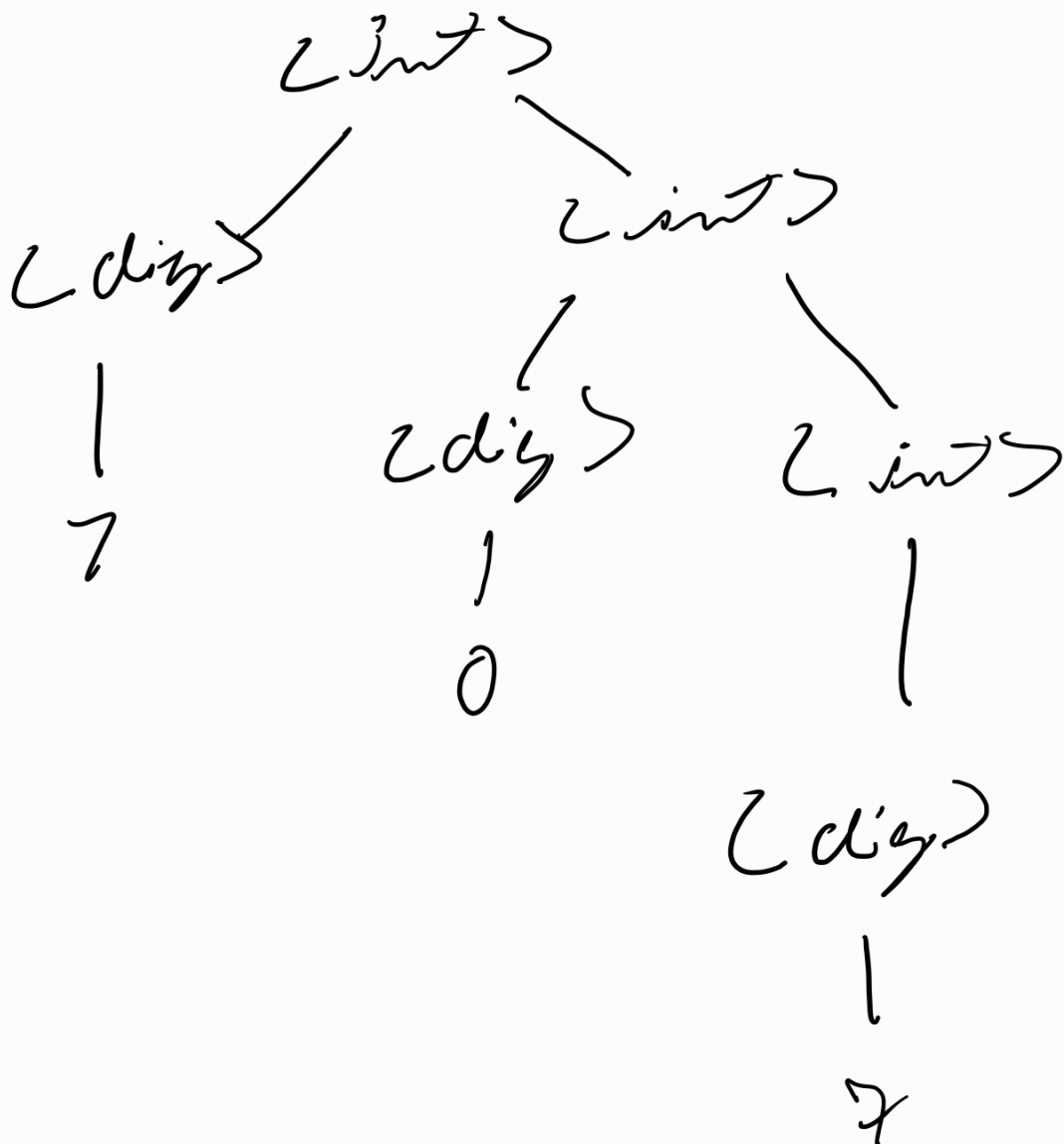
• Árvores de derivação.

Regras da gramática:

$\langle \text{int} \rangle \rightarrow \langle \text{dig} \rangle \langle \text{int} \rangle \mid \langle \text{dig} \rangle$

$\langle \text{dig} \rangle \rightarrow 0 \mid 1 \mid 2 \mid \dots \mid 9$

Prova: 107



Fixação da árvore

$\langle \text{exp} \rangle \rightarrow \langle \text{exp} \rangle + \langle \text{ter} \rangle \mid \langle \text{exp} \rangle * \langle \text{ter} \rangle \mid \langle \text{ter} \rangle$

$\langle \text{ter} \rangle \rightarrow 0 \mid 1 \mid 2 \mid 3 \dots \mid 9 \mid (\langle \text{exp} \rangle)$

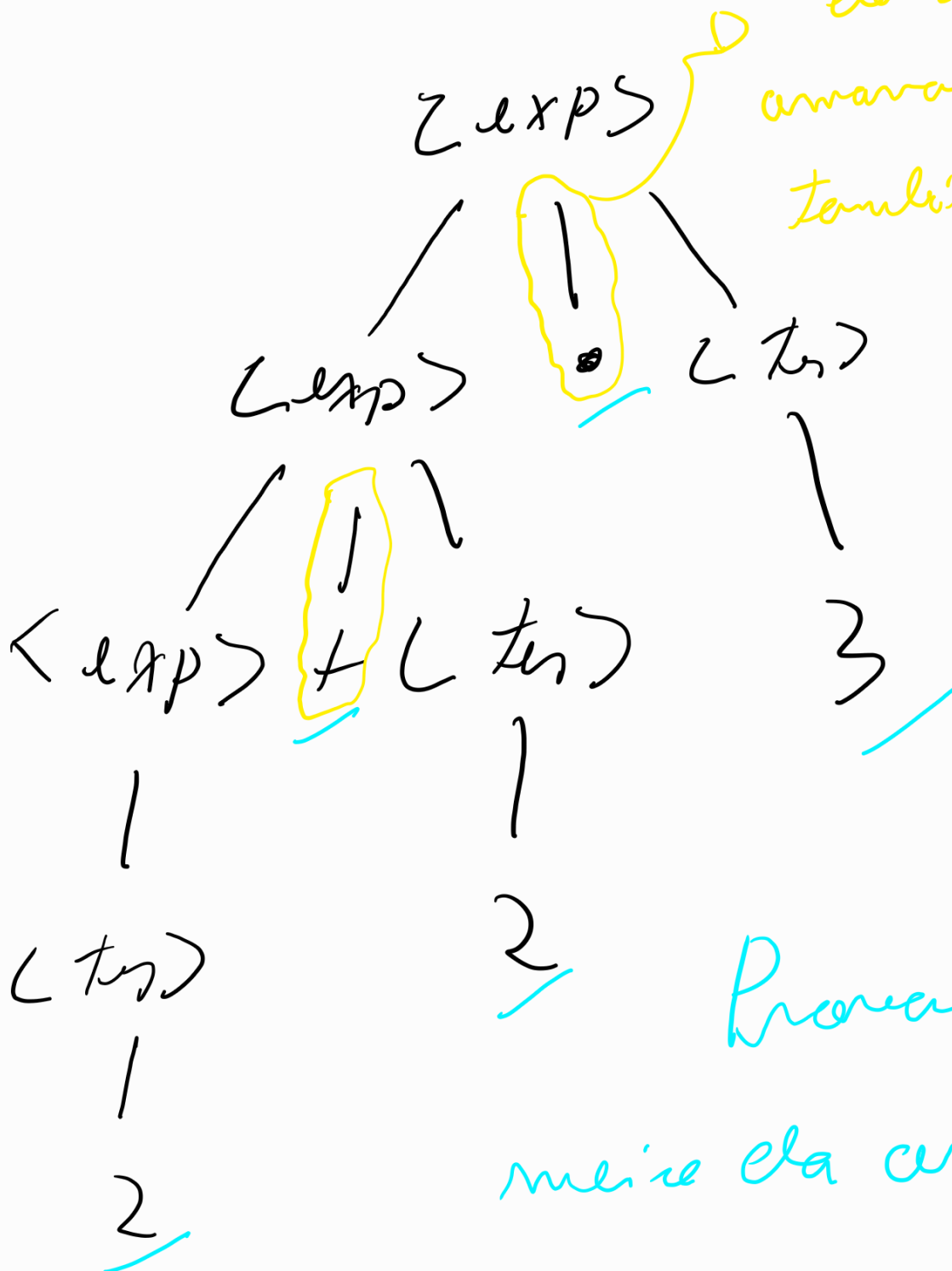
por ex: $2 + 2 * 3$

Expressões também

devem ser

analisadas, pois

também são símbolos



Proceder por

meio da árvore.

Prove: $2 \cdot 2 \neq 3$

$\langle \text{exp} \rangle$

$\langle \text{exp} \rangle + \langle \text{ter} \rangle$

$\langle \text{exp} \rangle$ $\cdot$ $\langle \text{ter} \rangle$ 3

$\mid$

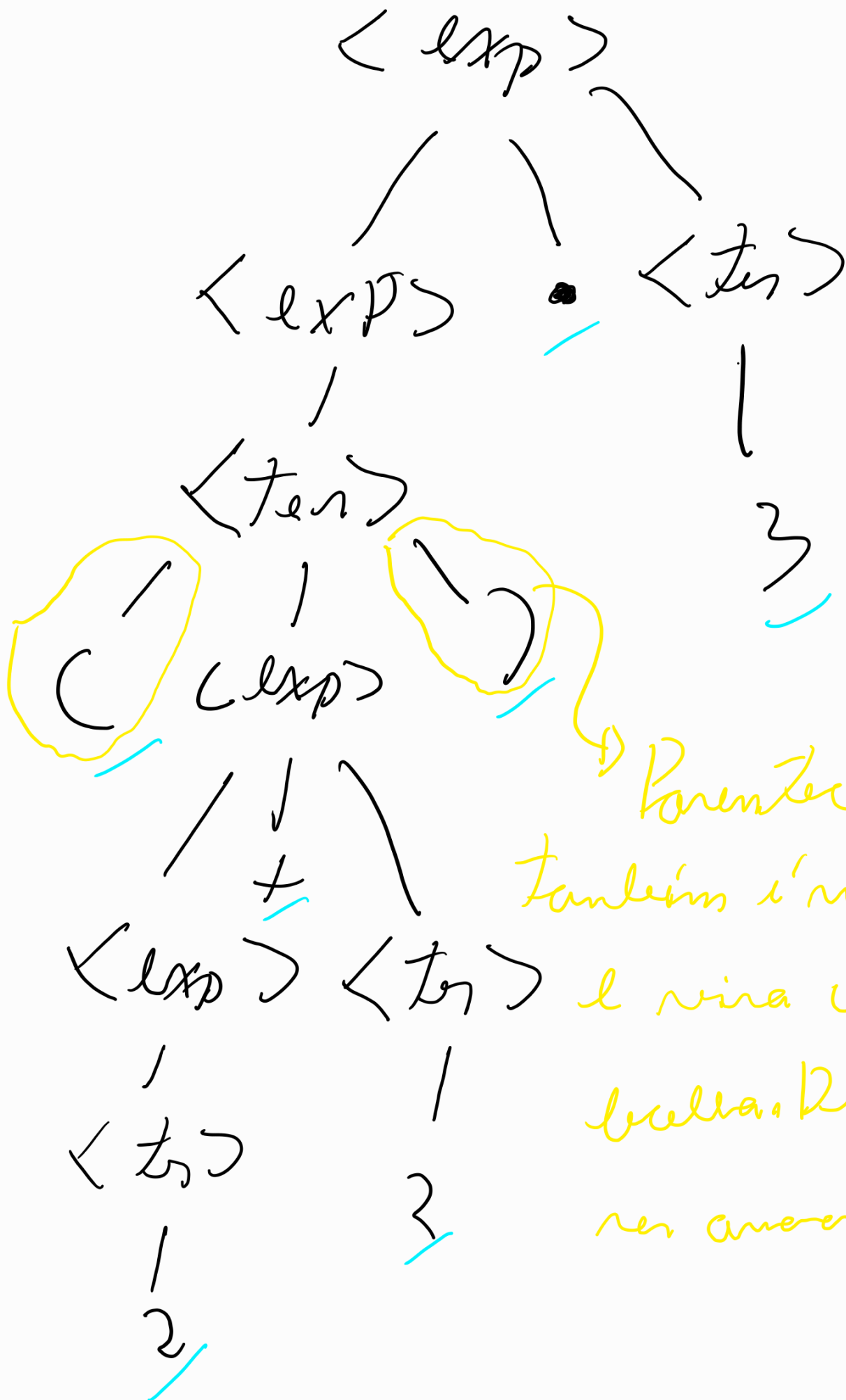
$\langle \text{ter} \rangle$

$\mid$

2

2

Prove $(2+2) \cdot 3$



Parentese
Também é incluído
e virá uma
bullet. Ruy
res anexo.

OBS: Nas árvores não há
divisão de esquerda e
direita.

Siguiência das árvores

Prova que é uma
chamada recursiva.

$\langle exp \rangle \rightarrow \langle exp \rangle + \langle ter \rangle \mid \langle exp \rangle * \langle ter \rangle \mid \langle ter \rangle$

$\langle ter \rangle \rightarrow 0 \mid 1 \mid 2 \mid 3 \dots \mid 9 \mid (\langle exp \rangle)$

Precedência de operadores:

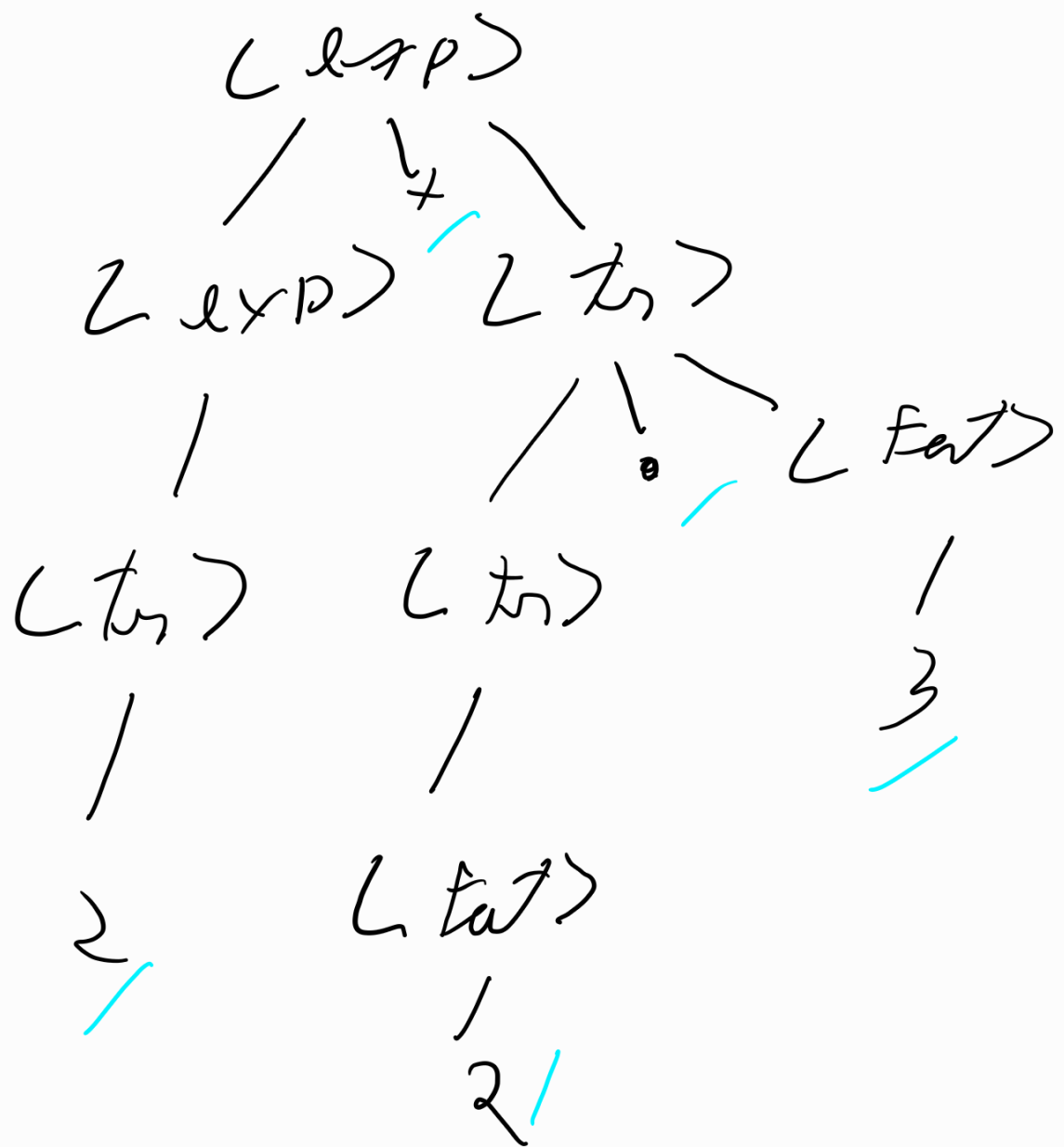
Ordem das operações
que devem ser feitas
primeiras nos cálculos.

Fiszerice cínere.

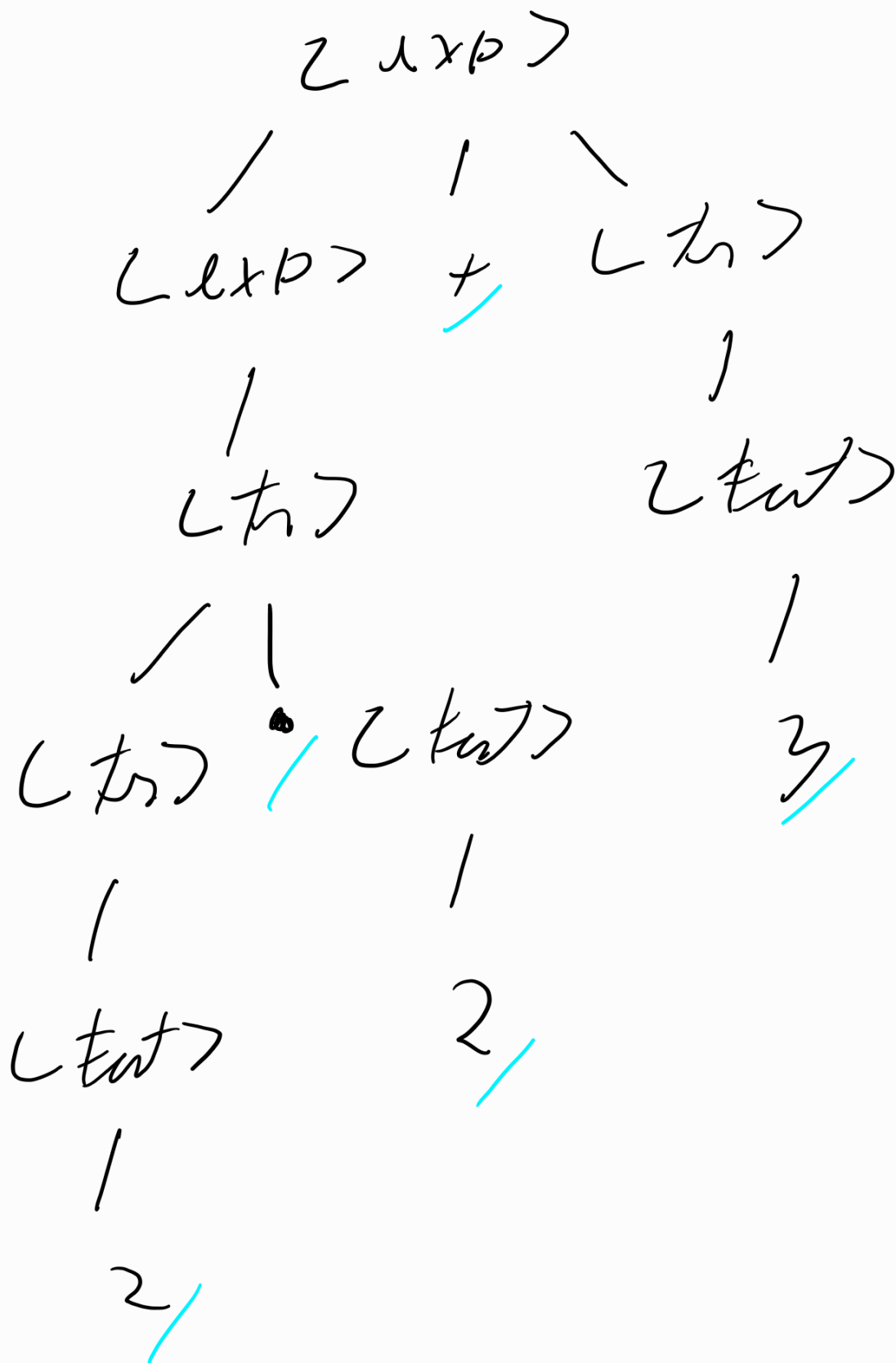
$$\begin{aligned} \langle \text{exp} \rangle &\rightarrow \langle \text{exp} \rangle + \langle \text{tr} \rangle / \langle \text{tr} \rangle \\ \langle \text{tr} \rangle &\rightarrow \langle \text{tr} \rangle \cdot \langle \text{fat} \rangle / \langle \text{fat} \rangle \\ \langle \text{fat} \rangle &\rightarrow 0 / 1 / 2 / \dots / \ell / \langle \text{exp} \rangle \end{aligned}$$

16. Nare tisminev'i

Prone: 2+2+3



Prove: $2 \cdot 2 + 3$



MUITO IMPORTANTE

OBS: Adicionamos mais um
nao terminal (Eut) e porque
isso? Por que dificultar a
anotação sem necessidade?

Porque agora temos precedência!

→ Conseguimos perceber que
agora os novos blocos novos
devem ter a ordem de execução
contra os operadores!

Obs: Quando temos duas
câmaras iguais como melhe-
mor qual bico?

Por Associação
da esquerda com a direita

A Associação é mais comum da
esquerda para a direita.

Lembres das palavras:

Precedência

e

Associação

